

```
-- =====
-- ECOMMERCE VIEW ASSIGNMENT (MYSQL)
-- COMPLETE STEP-BY-STEP SOLUTION
-- =====

-- =====
-- DATABASE SETUP
-- =====

CREATE DATABASE ecommerce_view_demo;
USE ecommerce_view_demo;

-- =====
-- TABLE CREATION
-- =====

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(100)
);

CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    order_date DATE,
    status VARCHAR(50),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- =====
-- SAMPLE DATA
-- =====
```

```
INSERT INTO customers (customer_name, email, city) VALUES
('Aung Aung', 'aung@gmail.com', 'Yangon'),
('Su Su', 'susu@gmail.com', 'Mandalay'),
('Kyaw Kyaw', 'kyaw@gmail.com', 'Yangon');
```

```
INSERT INTO products (product_name, category, price) VALUES
('Laptop', 'Electronics', 1500000),
('Phone', 'Electronics', 800000),
('Shoes', 'Fashion', 120000);
```

```
INSERT INTO orders (customer_id, order_date, status) VALUES
(1, '2024-01-10', 'Completed'),
(2, '2024-01-11', 'Pending'),
(1, '2024-01-12', 'Completed');
```

```
INSERT INTO order_items (order_id, product_id, quantity) VALUES
(1, 1, 1),
(1, 2, 2),
(2, 3, 1),
(3, 2, 1);
```

```
-- =====
-- Q1. VIEW: Customer basic information (hide email)
```

```
-- =====
CREATE VIEW customer_basic_info AS
SELECT customer_id, customer_name, city
FROM customers;
```

```
-- =====
-- Q2. VIEW: All completed orders
```

```
-- =====
CREATE VIEW completed_orders AS
SELECT *
FROM orders
WHERE status = 'Completed';
```

```
-- =====
-- Q3. VIEW: Order details (customer + product)
```

```
-- =====
CREATE VIEW order_details AS
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    p.category,
    oi.quantity,
    p.price
FROM orders o
```

```
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

```
-- =====
```

```
-- Q4. VIEW: Total sales per order
```

```
-- =====
```

```
CREATE VIEW order_total_sales AS
SELECT
    o.order_id,
    SUM(p.price * oi.quantity) AS total_sales
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY o.order_id;
```

```
-- =====
```

```
-- Q5. VIEW: High-value orders (>1,000,000)
```

```
-- =====
```

```
CREATE VIEW high_value_orders AS
SELECT *
FROM order_total_sales
WHERE total_sales > 1000000;
```

```
-- =====
```

```
-- Q6. VIEW: WITH CHECK OPTION
```

```
-- =====
```

```
CREATE VIEW completed_orders_secure AS
SELECT *
FROM orders
WHERE status = 'Completed'
WITH CHECK OPTION;
```

```
-- =====
```

```
-- Q7. UPDATE USING VIEW
```

```
-- =====
```

```
UPDATE customer_basic_info
SET city = 'Naypyidaw'
WHERE customer_id = 1;
```

```
-- =====
```

```
-- Q8. INVALID UPDATE (Should fail)
```

```
-- =====
```

```
-- This should fail:
-- UPDATE completed_orders_secure
-- SET status = 'Pending'
-- WHERE order_id = 1;
```

```
-- =====
```

-- Q9. VIEW: Customer purchase summary

```
=====
CREATE VIEW customer_purchase_summary AS
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS total_orders,
    SUM(p.price * oi.quantity) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY c.customer_id, c.customer_name;
```

-- Q10. VIEW: TEMPTABLE

```
=====
CREATE ALGORITHM = TEMPTABLE VIEW expensive_products AS
SELECT *
FROM products
WHERE price > 500000;
```

-- Q11. VIEW: Security (hide price)

```
=====
CREATE VIEW product_security_view AS
SELECT
    product_id,
    product_name,
    category
FROM products;
```

-- Q12. SHOW ALL VIEWS

```
=====
SHOW FULL TABLES
WHERE TABLE_TYPE = 'VIEW';
```

-- Q13. SHOW VIEW DEFINITION

```
=====
SHOW CREATE VIEW order_details;
```

-- Q14. DROP A VIEW

```
=====
DROP VIEW IF EXISTS expensive_products;
```

```
-- ADDITIONAL QUESTION 1:
```

```
-- Pending orders only
```

```
-- =====
```

```
CREATE VIEW pending_orders AS
```

```
SELECT *
```

```
FROM orders
```

```
WHERE status = 'Pending';
```

```
-- =====
```

```
-- ADDITIONAL QUESTION 2:
```

```
-- Top 3 expensive products
```

```
-- =====
```

```
CREATE VIEW top_3_expensive_products AS
```

```
SELECT *
```

```
FROM products
```

```
ORDER BY price DESC
```

```
LIMIT 3;
```

```
-- =====
```

```
-- ADDITIONAL QUESTION 3:
```

```
-- Customer orders by city
```

```
-- =====
```

```
CREATE VIEW customer_orders_by_city AS
```

```
SELECT
```

```
    c.city,
```

```
    c.customer_name,
```

```
    o.order_id,
```

```
    o.order_date,
```

```
    o.status
```

```
FROM customers c
```

```
JOIN orders o ON c.customer_id = o.customer_id;
```

```
-- =====
```

```
-- ADDITIONAL QUESTION 4:
```

```
-- Monthly sales report
```

```
-- =====
```

```
CREATE VIEW monthly_sales_report AS
```

```
SELECT
```

```
    DATE_FORMAT(o.order_date, '%Y-%m') AS sales_month,
```

```
    SUM(p.price * oi.quantity) AS monthly_sales
```

```
FROM orders o
```

```
JOIN order_items oi ON o.order_id = oi.order_id
```

```
JOIN products p ON oi.product_id = p.product_id
```

```
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m');
```

```
-- =====
```

```
-- ADDITIONAL QUESTION 5:
```

```
-- Products not ordered
```

```
-- =====
```

```

CREATE VIEW products_not_ordered AS
SELECT *
FROM products
WHERE product_id NOT IN (
    SELECT DISTINCT product_id
    FROM order_items
);

```

```

-- =====
-- TEST ALL VIEWS
-- =====

```

```

SELECT * FROM customer_basic_info;
SELECT * FROM completed_orders;
SELECT * FROM order_details;
SELECT * FROM order_total_sales;
SELECT * FROM high_value_orders;
SELECT * FROM customer_purchase_summary;
SELECT * FROM pending_orders;
SELECT * FROM top_3_expensive_products;
SELECT * FROM customer_orders_by_city;
SELECT * FROM monthly_sales_report;
SELECT * FROM products_not_ordered;

```

```

-- =====
-- END OF ECOMMERCE VIEW ASSIGNMENT
-- =====

```

The screenshot shows a SQL IDE interface with two query windows. The first window displays the execution of a query to show all views, resulting in a table with 11 rows and 2 columns: 'Tables_in_ecommerce_view_demo' and 'Table_type'. The second window displays the execution of a query to show the definition of the 'order_details' view, resulting in a table with 1 row and 3 columns: 'View', 'Create View', and 'collation_connection'.

Tables_in_ecommerce_view_demo	Table_type
completed_orders	VIEW
completed_orders_secure	VIEW
customer_basic_info	VIEW
customer_purchase_summary	VIEW
expensive_products	VIEW
high_value_orders	VIEW
order_details	VIEW
order_total_sales	VIEW
product_security_view	VIEW

View	Create View	collation_connection
order_details	CREATE ALGORITHM=UNDEFINED DEFINER='r...' utf8mb4	utf8mb4_0900_ai_ci

```

135 • UPDATE customer_basic_info
136 SET city = 'Naypyidaw'
137 WHERE customer_id = 1;
138 • SELECT * FROM customer_basic_info;
139 -- =====

```

customer_id	customer_name	city
1	Aung Aung	Naypyidaw
2	Su Su	Mandalay
3	Kyaw Kyaw	Yangon

```

147 -- =====
148 -- Q9. VIEW: Customer purchase summary
149 -- =====
150 • CREATE VIEW customer_purchase_summary AS
151 SELECT
152     c.customer_id,
153     c.customer_name,
154     COUNT(o.order_id) AS total_orders,
155     SUM(p.price * oi.quantity) AS total_spent
156 FROM customers c
157 JOIN orders o ON c.customer_id = o.customer_id
158 JOIN order_items oi ON o.order_id = oi.order_id
159 JOIN products p ON oi.product_id = p.product_id
160 GROUP BY c.customer_id, c.customer_name;
161 • select * from customer_purchase_summary;

```

customer_id	customer_name	total_orders	total_spent
1	Aung Aung	3	3900000.00
2	Su Su	1	120000.00

```

163 -- Q10. VIEW: TEMPTABLE
164 -- =====
165 • CREATE ALGORITHM = TEMPTABLE VIEW expensive_products AS
166 SELECT *
167 FROM products
168 WHERE price > 500000;
169
170 • select * from expensive_products;

```

product_id	product_name	category	price
1	Laptop	Electronics	1500000.00
2	Phone	Electronics	800000.00

```

172 -- Q11. VIEW: Security (hide price)
173 -- =====
174 • CREATE VIEW product_security_view AS
175 SELECT
176     product_id,
177     product_name,
178     category
179 FROM products;
180
181 • select * from product_security_view;
182 -- =====
200 -- Pending orders only
201 -- =====
202 • CREATE VIEW pending_orders AS
203 SELECT *
204 FROM orders
205 WHERE status = 'Pending';
206
207 • select * from pending_orders;
208 -- =====
210 -- Top 3 expensive products
211 -- =====
212 • CREATE VIEW top_3_expensive_products AS
213 SELECT *
214 FROM products
215 ORDER BY price DESC
216 LIMIT 3;
217
218 • select * from top_3_expensive_products;
219

```

product_id	product_name	category	price
1	Laptop	Electronics	1500000.00
2	Phone	Electronics	800000.00
3	Shoes	Fashion	120000.00

```

222  -- Customer orders by city
223  -- =====
224  • CREATE VIEW customer_orders_by_city AS
225  SELECT
226      c.city,
227      c.customer_name,
228      o.order_id,
229      o.order_date,
230      o.status
231  FROM customers c
232  JOIN orders o ON c.customer_id = o.customer_id;
233
234  • select * from customer_orders_by_city;

```

Result Grid

	sales_month	monthly_sales
2024-01		4020000.00

```

238  -- Monthly sales report
239  -- =====
240  • CREATE VIEW monthly_sales_report AS
241  SELECT
242      DATE_FORMAT(o.order_date, '%Y-%m') AS sales_month,
243      SUM(p.price * oi.quantity) AS monthly_sales
244  FROM orders o
245  JOIN order_items oi ON o.order_id = oi.order_id
246  JOIN products p ON oi.product_id = p.product_id
247  GROUP BY DATE_FORMAT(o.order_date, '%Y-%m');
248
249  • select * from monthly_sales_report;

```

Result Grid

product_id	product_name	category	price

```

253  -- Products not ordered
254  -- =====
255  • CREATE VIEW products_not_ordered AS
256  SELECT *
257  FROM products
258  WHERE product_id NOT IN (
259      SELECT DISTINCT product_id
260      FROM order_items
261  );
262
263  • select * from products_not_ordered;
264  -- =====

```

Result Grid

product_id	product_name	category	price